

Cours Deep Learning - Semaines 1 & 2

Thèse: Attaques d'extraction et défenses pour les DNN au-delà de ReLU

Tidiane DIALLO

Janvier 2026

- [SEMAINE 1 : Perceptron et Réseaux Multicouches \(MLP\)](#)
 - [Introduction aux Réseaux de Neurones](#)
 - [Contexte Historique](#)
 - [Pourquoi c'est important pour ta thèse](#)
 - [Le Neurone Biologique vs Artificiel](#)
 - [Le Perceptron Simple](#)
 - [Définition Mathématique](#)
 - [Fonction d'Activation du Perceptron Original](#)
 - [Interprétation Géométrique](#)
 - [Algorithme d'Apprentissage du Perceptron](#)
 - [Limitation : Le Problème XOR](#)
 - [Perceptron Multicouches \(MLP\)](#)
 - [Architecture](#)
 - [Notation Mathématique](#)
 - [Théorème d'Approximation Universelle](#)
 - [Propagation Avant \(Forward Pass\)](#)
 - [Algorithme](#)
 - [Importance pour l'Extraction de Modèles](#)
 - [Rétropropagation \(Backpropagation\)](#)
 - [Principe](#)
 - [La Règle de la Chaîne](#)
 - [Algorithme de Backpropagation](#)
 - [Lien avec la Cryptanalyse](#)
- [SEMAINE 2 : Fonctions d'Activation](#)
 - [Introduction](#)
 - [Rôle des Fonctions d'Activation](#)
 - [Importance Critique pour ta Thèse](#)
 - [Fonctions d'Activation Classiques](#)
 - [Sigmoid \(Logistique\)](#)
 - [Tanh \(Tangente Hyperbolique\)](#)
 - [ReLU et ses Variantes](#)
 - [ReLU \(Rectified Linear Unit\)](#)
 - [Propriété Cruciale : Linéaire par Morceaux](#)
 - [Variantes](#)
 - [GELU - Gaussian Error Linear Unit](#)
 - [Définition](#)
 - [Interprétation Probabiliste](#)
 - [Approximation Pratique \(GPT, BERT\)](#)
 - [Dérivée de GELU](#)
 - [Utilisation](#)
 - [SiLU/Swish](#)
 - [Définition](#)
 - [Dérivée de SiLU](#)
 - [Utilisation](#)
 - [Analyse Comparative](#)
 - [Tableau Récapitulatif](#)
 - [Dérivées Secondes](#)
 - [Implications pour l'Extraction de Modèles](#)
 - [Pourquoi ReLU est Vulnérable](#)
 - [Pourquoi GELU/SiLU sont Plus Résistants](#)
 - [Défis pour ta Thèse](#)
- [Résumé](#)
 - [Concepts Clés](#)
 - [Message Central](#)
- [Ressources](#)
 - [Vidéos Recommandées](#)
 - [Lectures](#)
 - [Code](#)

SEMAINE 1 : Perceptron et Réseaux Multicouches (MLP)

Introduction aux Réseaux de Neurones

Contexte Historique

Les réseaux de neurones artificiels s’inspirent du fonctionnement du cerveau humain. L’histoire commence en **1943** avec le modèle de McCulloch et Pitts, suivi du **perceptron de Rosenblatt en 1958**.

La révolution majeure arrive en **1986** quand **Rumelhart, Hinton et Williams** introduisent l’algorithme de rétropropagation (backpropagation), permettant l’entraînement efficace des réseaux multicouches.

Pourquoi c’est important pour ta thèse

Les attaques d’extraction de modèles exploitent la structure mathématique des réseaux de neurones. Comprendre :

- Comment les poids transforment les données
- Comment les gradients révèlent de l’information
- Pourquoi certaines activations (ReLU) créent des “points critiques”

...est **fondamental** pour comprendre les attaques cryptanalytiques sur les DNN.

Le Neurone Biologique vs Artificiel

Neurone Biologique	Neurone Artificiel
Dendrites (entrées)	$(x_1, x_2, \dots, x_n)$ (inputs)
Corps cellulaire	Somme pondérée + biais
Axone (sortie)	Fonction d’activation $\rightarrow y$
Synapses (connexions)	Poids $(w_1, w_2, \dots, w_n)$

Le Perceptron Simple

Définition Mathématique

Le perceptron est le modèle le plus simple d’un neurone artificiel.

Équation fondamentale :

$$z = \sum_{i=1}^n w_i \cdot x_i + b = \mathbf{w}^T \mathbf{x} + b$$

$$y = f(z)$$

où :

- $\mathbf{x}$ = vecteur d’entrée (features)
- $\mathbf{w}$ = vecteur de poids (weights)
- b = biais (bias)
- f = fonction d’activation
- y = sortie (output)

Fonction d’Activation du Perceptron Original

Le perceptron original utilise une **fonction de seuil (step function)** :

$$f(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{si } z < 0 \end{cases}$$

Interprétation Géométrique

Le perceptron définit un **hyperplan** dans l’espace des entrées :

$$\mathbf{w}^T \mathbf{x} + b = 0$$

Cet hyperplan **sépare** l’espace en deux régions :

- Points où $\mathbf{w}^T \mathbf{x} + b > 0 \rightarrow$ classe 1
- Points où $\mathbf{w}^T \mathbf{x} + b < 0 \rightarrow$ classe 0

Important pour ta thèse : Cette séparation linéaire est la base des “régions linéaires” créées par ReLU !

Algorithme d’Apprentissage du Perceptron

```
# Pseudo-code de l'algorithme du perceptron
pour chaque époque:
    pour chaque exemple (x, y_vrai):
        y_pred = 1 si (w·x + b) >= 0 sinon 0
```

```

erreur = y_vrai - y_pred
# Mise à jour des poids
w = w + η * erreur * x
b = b + η * erreur

```

où η (eta) est le **taux d'apprentissage** (learning rate).

Limitation : Le Problème XOR

Le perceptron simple ne peut résoudre que les problèmes **linéairement séparables**.

Le XOR n'est pas linéairement séparable et nécessite **plusieurs couches** → d'où le MLP !

Perceptron Multicouches (MLP)

Architecture

Un MLP est un réseau **feedforward** composé de :

- **Couche d'entrée** : reçoit les features
- **Couches cachées** : transformations non-linéaires
- **Couche de sortie** : produit la prédiction

Notation Mathématique

Pour un MLP avec L couches :

Couche 1 :

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{a}^{[l]} = f^{[l]}(\mathbf{z}^{[l]})$$

où :

- $\mathbf{W}^{[l]}$ = matrice de poids de la couche l (dimensions : $n_l \times n_{l-1}$)
- $\mathbf{b}^{[l]}$ = vecteur de biais de la couche l
- $\mathbf{z}^{[l]}$ = pré-activation (avant la fonction d'activation)
- $\mathbf{a}^{[l]}$ = activation (après la fonction d'activation)
- $f^{[l]}$ = fonction d'activation de la couche l

Convention : $\mathbf{a}^{[0]} = \mathbf{x}$ (l'entrée)

Théorème d'Approximation Universelle

Théorème (Cybenko, 1989) :

Un MLP avec une seule couche cachée et suffisamment de neurones peut approximer n'importe quelle fonction continue sur un compact.

En pratique, des réseaux **profonds** (plusieurs couches) sont plus efficaces que des réseaux larges (beaucoup de neurones dans une seule couche).

Propagation Avant (Forward Pass)

Algorithme

Le forward pass calcule la sortie du réseau étape par étape :

```

def forward_pass(x, weights, biases, activations):
    a = x # activation initiale = entrée
    for l in range(len(weights)):
        z = weights[l] @ a + biases[l] # pré-activation
        a = activations[l](z) # activation
    return a # sortie finale

```

Importance pour l'Extraction de Modèles

Lors d'une attaque d'extraction :

1. L'attaquant envoie des **requêtes** (inputs x)
2. Il observe les **réponses** (outputs y)
3. Il essaie de **reconstruire** les poids W et biais b

Point clé : Avec ReLU, les "points critiques" (où ReLU change de comportement) révèlent de l'information sur les

poids !

Rétropropagation (Backpropagation)

Principe

La rétropropagation calcule les **gradients** de la fonction de perte par rapport à chaque paramètre, en utilisant la **règle de la chaîne** (chain rule).

Objectif : Minimiser la fonction de perte $\mathcal{L}(\hat{y}, y)$

Méthode : Descente de gradient

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

La Règle de la Chaîne

Si $y = f(g(x))$, alors :

$$\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$$

Pour un réseau de neurones :

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Algorithme de Backpropagation

- FORWARD PASS :** Calculer toutes les activations $a^{[l]}$
- CALCUL DE LA PERTE :** $\mathcal{L} = \text{Loss}(y_{\text{pred}}, y_{\text{vrai}})$
- BACKWARD PASS :**
 - Calculer $\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial z^{[L]}}$ (couche de sortie)
 - Pour $l = L-1$ jusqu'à 1 : $\delta^{[l]} = (W^{[l+1]})^T \delta^{[l+1]} \odot f'(z^{[l]})$
- GRADIENTS :**
 - $\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T$
 - $\frac{\partial \mathcal{L}}{\partial b^{[l]}} = \delta^{[l]}$
- MISE À JOUR :**
 - $W^{[l]} = W^{[l]} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W^{[l]}}$
 - $b^{[l]} = b^{[l]} - \eta \cdot \frac{\partial \mathcal{L}}{\partial b^{[l]}}$

Lien avec la Cryptanalyse

Observation clé de Carlini et al. (CRYPTO 2020) :

En utilisant des **différences finies** (numerical differentiation), un attaquant peut estimer les gradients sans accès direct au modèle :

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + \epsilon e_i) - f(x - \epsilon e_i)}{2\epsilon}$$

Ces gradients révèlent de l'information sur la structure du réseau !

SEMAINE 2 : Fonctions d'Activation

Introduction

Rôle des Fonctions d'Activation

Les fonctions d'activation introduisent la **non-linéarité** dans les réseaux de neurones. Sans elles, un réseau profond serait équivalent à une simple transformation linéaire.

Importance Critique pour ta Thèse

Le choix de l'activation détermine si le réseau est vulnérable aux attaques d'extraction !

- ReLU** → Crée des régions **linéaires par morceaux** → Attaques efficaces (CRYPTO 2020)
- GELU/SiLU** → Fonctions **lisses** → Attaques beaucoup plus difficiles

C'est le **cœur de ton sujet de thèse** !

Fonctions d'Activation Classiques

Sigmoid (Logistique)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Dérivée :

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Propriétés :

- Sortie dans $(0, 1)$ → interprétation probabiliste
- Problème** : Gradient s’annule aux extrêmes (saturation)
- Maximum de $\sigma'(x) = 0.25$ à $(x = 0)$

Tanh (Tangente Hyperbolique)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$

Dérivée :

$$\tanh'(x) = 1 - \tanh^2(x)$$

Avantage sur sigmoid : Sorties centrées → meilleure convergence

ReLU et ses Variantes

ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{si } x > 0 \\ 0 & \text{si } x \leq 0 \end{cases}$$

Dérivée :

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{si } x > 0 \\ 0 & \text{si } x < 0 \end{cases}$$

Propriété Cruciale : Linéaire par Morceaux

C’est LA propriété qui rend ReLU vulnérable aux attaques !

Un réseau avec ReLU divise l’espace d’entrée en **régions linéaires** :

- Chaque neurone ReLU crée un hyperplan : $(w^T x + b = 0)$
- De part et d’autre de l’hyperplan, le neurone est soit actif (x) soit inactif (0)
- Avec (n) neurones, on peut avoir jusqu’à (2^n) régions !

Implication pour l’extraction :

Les points où le comportement change (hyperplans) révèlent les poids du réseau !

Variantes

- Leaky ReLU** : $f(x) = \max(\alpha x, x)$ avec $(\alpha = 0.01)$
- PReLU** : (α) est appris
- ELU** : $f(x) = x$ si $(x > 0)$, sinon $(\alpha(e^x - 1))$

GELU - Gaussian Error Linear Unit

Définition

Paper original : Hendrycks & Gimpel (2016)

$$\text{GELU}(x) = x \cdot \Phi(x)$$

où $(\Phi(x))$ est la **fonction de répartition** (CDF) de la loi normale standard :

$$\Phi(x) = \frac{1}{2} \left[1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right]$$

Interprétation Probabiliste

GELU peut être vu comme une **porte stochastique** :

$$\text{GELU}(x) = x \cdot P(X \leq x) \text{ où } X \sim \mathcal{N}(0, 1)$$

Approximation Pratique (GPT, BERT)

$$\text{GELU}(x) \approx 0.5x \left[1 + \tanh \left(\sqrt{\frac{2}{\pi}} \left(x + 0.044715x^3 \right) \right) \right]$$

Dérivée de GELU

$$\text{GELU}'(x) = \Phi(x) + x \cdot \phi(x)$$

où $\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$ est la PDF normale.

Utilisation

GELU est utilisé dans :

- **BERT** (Bidirectional Encoder Representations from Transformers)
- **GPT** (Generative Pre-trained Transformer)
- **Vision Transformers (ViT)**

SiLU/Swish

Définition

Paper original : Ramachandran et al. (2017)

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

Dérivée de SiLU

$$\text{SiLU}'(x) = \sigma(x)[1 + x(1 - \sigma(x))]$$

Utilisation

SiLU est populaire dans :

- **EfficientNet** (modèles de vision)
- **YOLOv5/v8** (détection d’objets)

Analyse Comparative

Tableau Récapitulatif

Activation	Formule	Lisse ? Piecewise Linear ?	
Sigmoid	$\frac{1}{1+e^{-x}}$	Oui	Non
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	Oui	Non
ReLU	$\max(0,x)$	Non	Oui
GELU	$x\Phi(x)$	Oui	Non
SiLU	$x\sigma(x)$	Oui	Non

Dérivées Secondes

Activation	Dérivée seconde	Points critiques
ReLU	0 partout	Oui, détectables
GELU	Non nulle	Non
SiLU	Non nulle	Non

Implications pour l’Extraction de Modèles

Pourquoi ReLU est Vulnérable

L’attaque CRYPTO 2020 exploite la structure piecewise-linear de ReLU :

1. **Points critiques** : Aux hyperplans $w^T x + b = 0$, le comportement change brutalement
2. **Détection des changements** : Les dérivées numériques révèlent ces points
3. **Extraction des poids** : La différence de gradients révèle les poids

Pourquoi GELU/SiLU sont Plus Résistants

ReLU	GELU/SiLU
Changement discret	Transition continue
Dérivée = 0 ou 1	Dérivée varie continûment

Défis pour ta Thèse

Question de recherche centrale :

Comment adapter les attaques cryptanalytiques pour extraire des modèles utilisant GELU/SiLU ?

Pistes possibles :

- 1. Approximation piecewise-linear de GELU
- 2. Utilisation des points d’inflexion
- 3. Méthodes d’optimisation alternatives
- 4. Nouvelles approches d’attaque

Résumé

Concepts Clés

Concept	Point clé	Importance pour la thèse
Perceptron	$y = f(w^T x + b)$	Base de tout neurone
Forward pass	$a^{[l]} = f(W^{[l]} a^{[l-1]} + b^{[l]})$	Ce que l’attaquant observe
Backpropagation	Chain rule	Révèle la structure
ReLU	Piecewise linear	Vulnérable
GELU/SiLU	Lisses	Plus résistants

Message Central

La nature **piecewise-linear** de ReLU crée des “signatures” détectables par les attaques cryptanalytiques. Les activations lisses comme GELU et SiLU masquent ces signatures, rendant l’extraction beaucoup plus difficile.

C’est le défi que ta thèse doit résoudre !

Ressources

Vidéos Recommandées

- 1. **3Blue1Brown - Neural Networks** : https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- 2. **Andrej Karpathy - Micrograd** : <https://www.youtube.com/watch?v=VMj-3S1tku0>
- 3. **fast.ai - Lesson 13** : <https://course.fast.ai/Lessons/lesson13.html>

Lectures

- 1. **Deep Learning Book** (Goodfellow et al.) : <https://www.deeplearningbook.org/>
- 2. **Paper GELU** : <https://arxiv.org/abs/1606.08415>
- 3. **Paper SiLU/Swish** : <https://arxiv.org/abs/1710.05941>

Code

- 1. **Micrograd** : <https://github.com/karpathy/micrograd>
- 2. **Ton projet** : `/root/these-tidiane-diallo/`